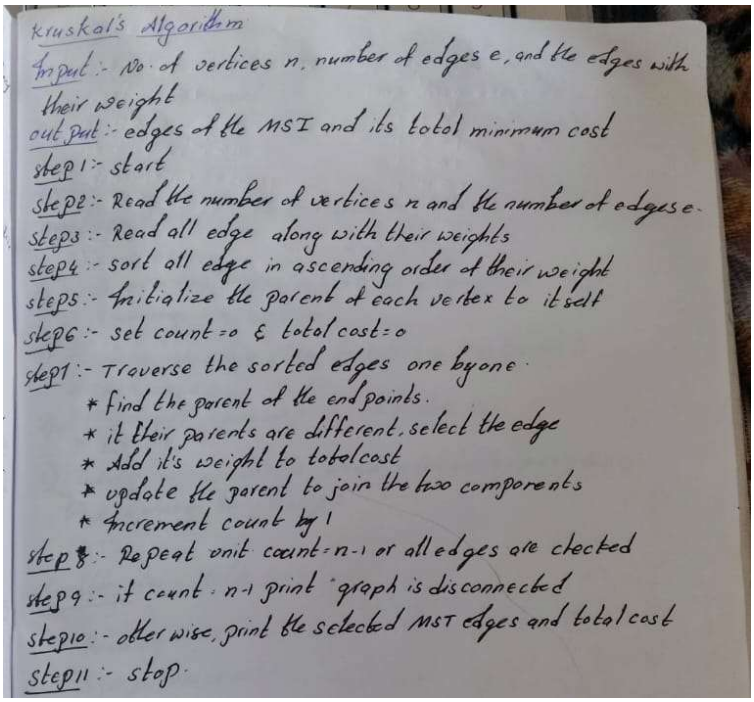
PRACTICAL: 10

AIM:- Implement Kruskal's algorithm.

Algorithms:-


Kruskal's Algorithm
Input :- No. of vertices n, number of edges e, and the edges with their weight
output :- edges of the MST and its total minimum cost
step1 :- start
step2 :- Read the number of vertices n and the number of edges e.
step3 :- Read all edge along with their weights
step4 :- sort all edge in ascending order of their weight
step5 :- initialize the parent of each vertex to itself
step6 :- set count=0 & total cost=0
step7 :- Traverse the sorted edges one by one
* find the parent of the end points.
* if their parents are different, select the edge
* Add it's weight to totalcost
* update the parent to join the two components
* increment count by 1
step8 :- Repeat until count=n-1 or all edges are checked
step9 :- if count : n-1 print "graph is disconnected
step10 :- otherwise, print the selected MST edges and total cost
step11 :- stop

Program/code:-

```
#include <iostream>
using namespace std;
struct Edge {
    int u, v, weight;
};
int main() {
int n, e;
```

```
Edge edge[100];
cout << "Enter number of vertices: ";
cin >> n;
cout << "Enter number of edges: ";
cin >> e;
cout << "Enter edges (u v weight):\n";
for (int i = 0; i < e; i++) {
cin >> edge[i].u >> edge[i].v >> edge[i].weight;
}
// Sort edges by weight
for (int i = 0; i < e- 1; i++) {
for (int j = 0; j < e- i- 1; j++) {
if (edge[j].weight > edge[j + 1].weight) {
Edge temp = edge[j];
edge[j] = edge[j + 1];
edge[j + 1] = temp;
}
}
}
int parent[10];
for (int i = 0; i < n; i++) {
parent[i] = i;
}
int count = 0, totalCost = 0;
```

```
cout << "\nEdges in Minimum Spanning Tree:\n";
for (int i = 0; i < e && count < n- 1; i++) {
int u = edge[i].u;
int v = edge[i].v;
int a = u, b = v;
while (parent[a] != a)
a =parent[a];
while (parent[b] != b)
b =parent[b];
if (a != b) {
cout << u << "- " << v
<< " : " << edge[i].weight << endl;
parent[a] = b;
totalCost += edge[i].weight;
count++;
}
}
if (count != n- 1) {
cout << "Graph is disconnected!\n";
} else {
cout << "Minimum Cost = " << totalCost << endl;
}
return 0;
}
```

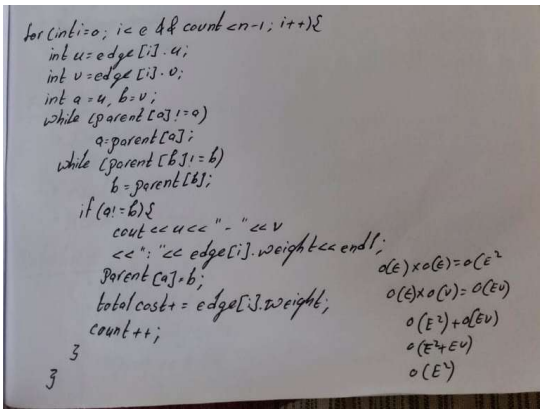
Output:-

```
Enter number of vertices: 4
Enter number of edges: 5
Enter edges (u v weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

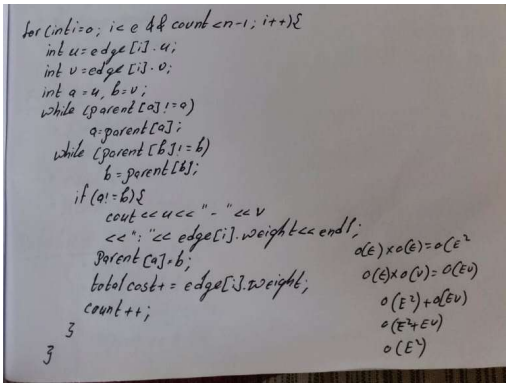
Edges in Minimum Spanning Tree:
2- 3 : 4
0- 3 : 5
0- 1 : 10
Minimum Cost = 19
```

Time complexity:-

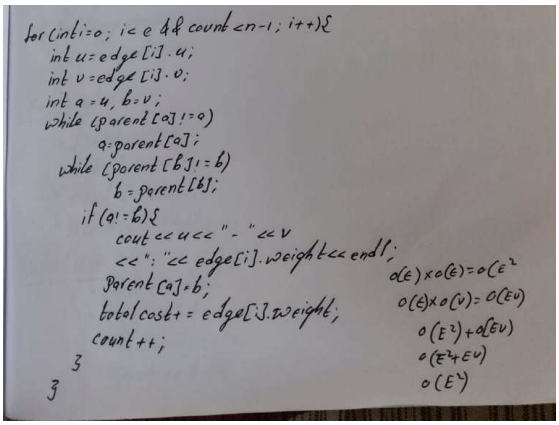
Best case:-



Worst case:-



Average case:-



Conclusion :-

Kruskal's algorithm finds the Minimum Spanning Tree (MST) of a connected, undirected, weighted graph by greedily picking the smallest edges and avoiding cycles using a Union-Find (Disjoint Set Union) data structure.